

Day 17 — Lifecycle Management & Version Control

Git integration, deployment pipelines, dev/test/prod stages, parameterization

Objective

Today you will connect a Fabric workspace to Git for version control, then build a deployment pipeline that promotes content through Development → Test → Production stages. This is how professional teams manage Fabric content safely, and it is a core “Implement & Manage” exam topic.

For Git integration you need either an Azure DevOps organization (free tier works) or a GitHub account. Set one up before starting if you don't have one. For deployment pipelines you need at least Fabric trial capacity.

Git integration is bidirectional. Commit pushes Fabric → Git. Update pulls Git → Fabric. This lets multiple engineers collaborate on the same workspace content with full history and rollback.

TASK 1 — Connect a Workspace to Git

Step 1. Create (or reuse) a Git repository in Azure DevOps or GitHub. Create an empty branch named “main” and a folder for Fabric items.

Step 2. In a Fabric workspace, open Workspace settings → Git integration.

Step 3. Connect to your repository: select the organization/account, project, repo, branch, and folder.

Step 4. Once connected, observe the source-control panel. Every supported item shows a Git status (sync'd, modified, conflict).

Step 5. Create a new item (e.g., a notebook). Notice it now shows as “Uncommitted” in the source control panel.

TASK 2 — Commit and Sync Changes

Step 1. Commit your new item to the repo: select it in the source-control panel → Commit. Add a commit message.

Step 2. Open your Git repo in the browser. Confirm the item now appears as files (JSON definitions + metadata).

Step 3. Make a change to the item inside Fabric. Observe it shows as “Modified”. Commit again.

Step 4. Now simulate a change made OUTSIDE Fabric: edit a file directly in the Git repo (or have a teammate do it) and commit. Back in Fabric, the source-control panel shows “Update available”. Click Update to pull the change into the workspace.

TASK 3 — Branching Strategy

Step 1. In your Git provider, create a feature branch named “feature/add-transform” off main.

Step 2. In Fabric, use “Branch out to new workspace” (or switch branch) to work on the feature branch in isolation.

Step 3. Make a change, commit it to the feature branch.

Step 4. In your notebook, describe the workflow: how would a team use feature branches + pull requests to review Fabric changes before merging to main?

TASK 4 — Build a Deployment Pipeline

Deployment pipelines promote content across stages without Git. They’re the Fabric-native way to move from Dev to Test to Prod.

Step 1. In Fabric, go to Deployment pipelines (from the left nav or workspace) → Create pipeline. Name it “Retail_Deployment”.

Step 2. The pipeline has three default stages: Development, Test, Production. Assign your current workspace to the Development stage.

Step 3. Create Test and Production workspaces and assign them to the corresponding stages.

Step 4. Add a few items to your Development workspace (a Lakehouse, a notebook, a pipeline).

Step 5. Click Deploy to push Development content to Test. Observe the comparison view that shows what will change (new, different, missing items).

Step 6. Deploy from Test to Production. Confirm the items now exist in all three stages.

TASK 5 — Deployment Rules and Parameterization

Different stages need different configurations — e.g., Test should point at a test data source, Prod at the production source. Deployment rules handle this.

Step 1. On the Production stage, configure a deployment rule for a data source: when content is deployed to Prod, the Lakehouse connection should switch to the production Lakehouse instead of the dev one.

Step 2. In your notebook, explain why you would NOT want dev and prod pointing at the same data source.

Step 3. Research parameterized deployment: how do you handle connection strings, workspace IDs, and environment-specific values across stages?

- ▶ What is the difference between Commit and Update in Fabric Git integration?
- ▶ What is the difference between Git integration and a Deployment pipeline? When do you use each?
- ▶ What do deployment rules solve? Give an example.
- ▶ How many stages does a Fabric deployment pipeline have by default, and what are they?
- ▶ Why should dev and prod stages point at different data sources?
- ▶ A workspace connected to Git with at least 2 commits

- ▶ A feature branch created and committed to
- ▶ A deployment pipeline “Retail_Deployment” with Dev/Test/Prod workspaces
- ▶ Content deployed through all three stages
- ▶ At least one deployment rule configured on the Production stage

Self-Check Questions Deliverables

Self-Check Questions

What is the difference between Commit and Update in Fabric Git integration?

What is the difference between Git integration and a Deployment pipeline? When do you use each?

What do deployment rules solve? Give an example.

How many stages does a Fabric deployment pipeline have by default, and what are they?

Why should dev and prod stages point at different data sources?

Deliverables

A workspace connected to Git with at least 2 commits

A feature branch created and committed to

A deployment pipeline “Retail_Deployment” with Dev/Test/Prod workspaces

Content deployed through all three stages

At least one deployment rule configured on the Production stage

